

Practical Tutorial: Custom Skill Creation

Building .agents/skills/gemma-eval/SKILL.md with Explicit YAML Frontmatter & Markdown Body

Step-by-Step Overview: This guide walks through the exact process of authoring a workspace custom skill in Google Antigravity. A skill comprises two critical components: (1) **YAML Frontmatter** defining the trigger contract used for discovery via progressive disclosure, and (2) **Markdown Body** containing actionable runbooks, helper scripts, and validation criteria.

Step 1: Create the Skill Directory Layout

Inside your project root, create the standard directory structure under .agents/skills/gemma-eval/:

```
# In Terminal (PowerShell or Bash) at Project Root:
mkdir -p .agents/skills/gemma-eval/scripts
mkdir -p .agents/skills/gemma-eval/references
```

Step 2: Author the YAML Frontmatter (The Trigger Contract)

Open .agents/skills/gemma-eval/SKILL.md. The file **MUST** begin on Line 1 with three dashes (---) and conclude the frontmatter with three dashes (---).

```
---
name: gemma-eval
description: >-
  Use this skill when the user asks to evaluate, benchmark, fine-tune, or test
  Gemma models (e.g., Gemma 2B, 7B, 9B, 27B), run automated model evaluations,
  compute perplexity/loss metrics, or validate inference prompt templates.
---
```

Frontmatter Key Guidelines:

- **name:** Must match the directory name (gemma-eval), lowercase and hyphenated.
- **description:** Written in third-person starting with 'Use this skill when...'. List specific target keywords, domain concepts, model variants, and command intents.

Step 3: Author the Markdown Body & Procedural Steps

Below the frontmatter, write the actionable Markdown runbook. Use clear numbered stages, relative links to helper scripts, and automated verification commands:

Gemma Model Evaluation & Benchmarking Runbook

This runbook guides the agent through pre-flight checks, benchmark execution, and metric analysis for Gemma checkpoints.

1. Environment Pre-Flight Check

Before starting, execute the pre-flight verification script to check GPU availability and dependencies:

```
- [verify_environment.py](./scripts/verify_environment.py)
```

2. Automated Benchmark Execution

Run the evaluation harness specifying model checkpoint, benchmark dataset, and batch configuration:

```
```bash
python .agents/skills/gemma-eval/scripts/evaluate_model.py \
 --model-id "google/gemma-2-9b-it" \
 --benchmark "mmlu-pro" \
 --batch-size 4 \
 --output-dir "./eval_results"
```
```

3. Metric Inspection & Validation

Inspect generated metrics in `./eval\_results/metrics.json` against standard baseline thresholds:

```
- [metrics_reference.md](./references/metrics_reference.md)
```

4. Verification Criteria

1. Confirm `./eval\_results/metrics.json` and `./eval\_results/eval\_summary.md` are present.
2. Ensure perplexity score is ≤ 8.5 and MMLU-Pro accuracy is $\geq 65.0\%$.
3. Confirm zero CUDA out-of-memory (OOM) exceptions in log output.

Step 4: Add Helper Scripts & Reference Manuals

Leverage **Progressive Disclosure** by isolating complex logic and extensive documentation into modular files:

- **scripts/verify_environment.py**: Python script checking PyTorch version, CUDA devices, and Hugging Face Transformers installation.
- **references/metrics_reference.md**: Reference tables containing baseline scores (MMLU, GSM8K, HumanEval) and troubleshooting procedures.

Step 5: Testing & Validating Skill Discovery

Once files are saved, Antigravity automatically indexes the skill upon your next message. You can test it by prompting:

'Evaluate the Gemma 2 9B model on MMLU-Pro and check perplexity.'

The agent will match the trigger description, dynamically load SKILL.md, inspect verify_environment.py, and execute the runbook.